In [11]:
```
# 1. Import libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.optimizers import Adam
```

In [12]:
```
# 2. Load and preprocess dataset
path = r"C:\Users\ombah\OmCore\BE Practical\LP-4 (Deep Learning)\LP IV Updated Datasets\Object Detection\caltech-101"


datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    validation_split=0.2
)

train_data = datagen.flow_from_directory(
    path,
    target_size=(224, 224),
    batch_size=32,
    class_mode="categorical",
    subset="training"
)

val_data = datagen.flow_from_directory(
    path,
    target_size=(224, 224),
    batch_size=32,
    class_mode="categorical",
    subset="validation"
)
```

```
Found 7356 images belonging to 102 classes.
Found 1788 images belonging to 102 classes.
```

In [3]:
```
# 3. Load pre-trained VGG16 model
base_model = VGG16(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)
```

In [4]:
```
# 4. Freeze base layers
base_model.trainable = False
```

In [5]:
```
# 5. Add custom classification layers
x = Flatten()(base_model.output)

x = Dense(256, activation="relu")(x)

x = Dropout(0.5)(x)

output = Dense(
    train_data.num_classes,
    activation="softmax"
)(x)

model = Model(base_model.input, output)
```

In [6]:
```
# 6. Compile model
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are ins
talled, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
```

In [7]:
```
# 7. Train the model
history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=2,
```

```
    steps_per_epoch=50,
    validation_steps=10
)
```

```
Epoch 1/2
50/50 ━━━━━━━━━━━━━━━━━━━━ 167s 3s/step - accuracy: 0.3119 - loss: 9.0261 - val_accuracy: 0.4250 - val_loss: 3.0441
Epoch 2/2
50/50 ━━━━━━━━━━━━━━━━━━━━ 198s 4s/step - accuracy: 0.3950 - loss: 3.1976 - val_accuracy: 0.4125 - val_loss: 3.0203
```

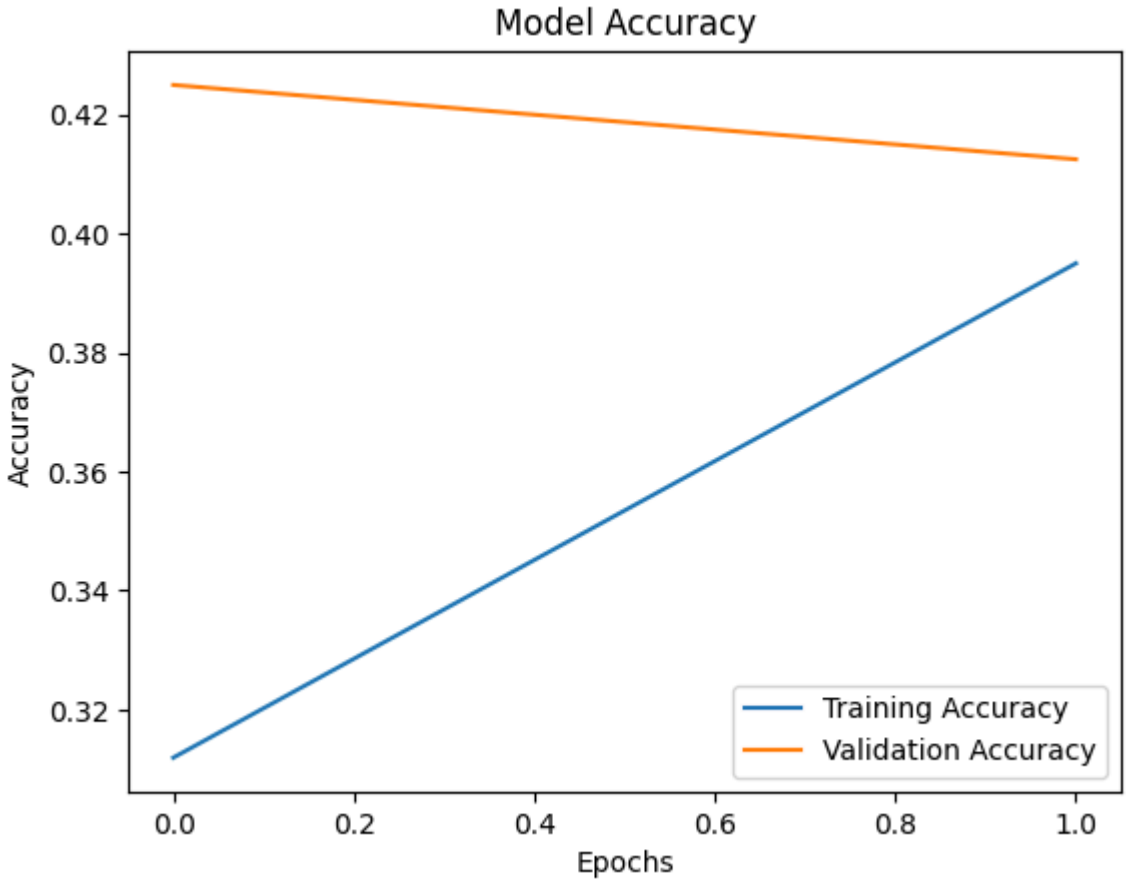
In [8]:
```
# 8. Evaluate the model
loss, accuracy = model.evaluate(val_data)

print("Validation Loss:", loss)
print("Validation Accuracy:", accuracy)
```

```
56/56 ━━━━━━━━━━━━━━━━━━━━ 192s 3s/step - accuracy: 0.4776 - loss: 2.6880
Validation Loss: 2.688002824783325
Validation Accuracy: 0.4776286482810974
```

In [15]:
```
# 9. Visualize training accuracy
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.title("Model Accuracy")
plt.legend()
plt.show()
```



In [22]:
```
#10. Visualize training loss
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Model Loss")
plt.legend()
plt.show()

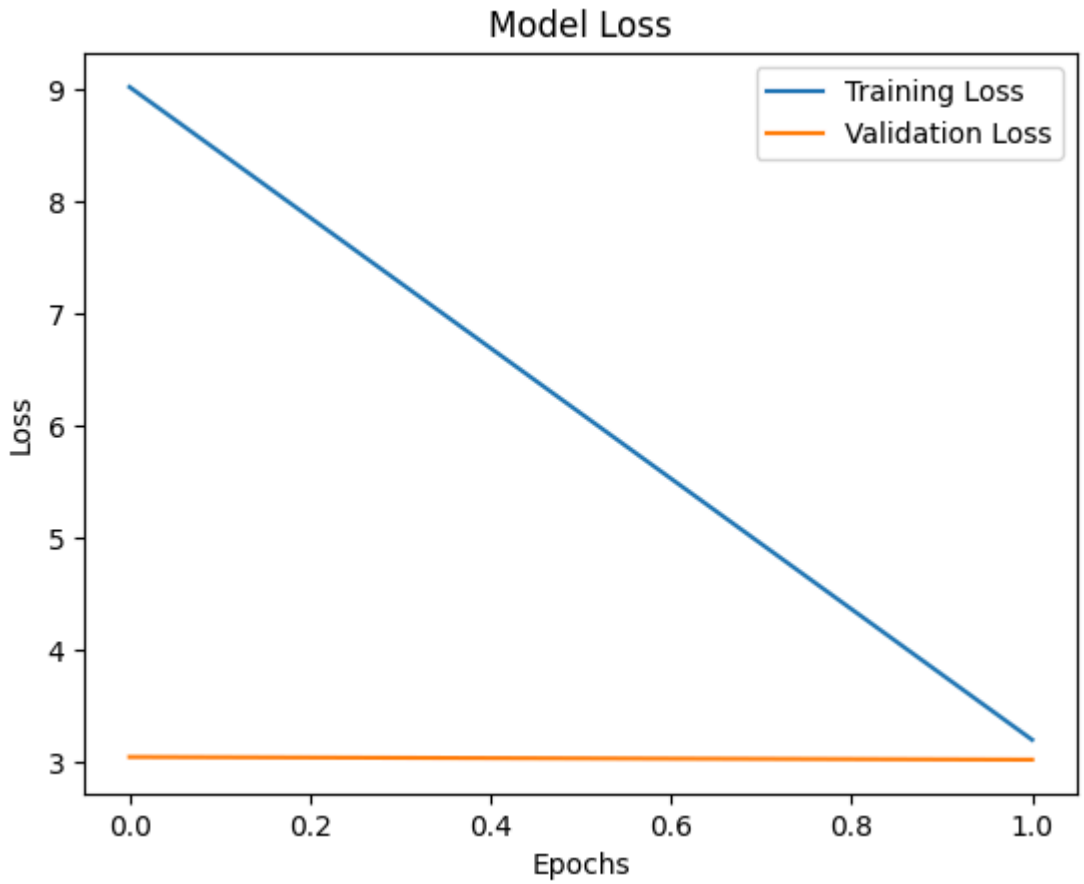
# 11. Visualize prediction
img, label = next(val_data)
```

```
prediction = model.predict(img[:1])

class_index = np.argmax(prediction[0])

class_names = list(train_data.class_indices.keys())

plt.imshow(img[0] / 255.0)
plt.title("Predicted: " + class_names[class_index])
plt.axis("off")
plt.show()
```




```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 161ms/step
```

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.485
0196..0.5923961].
```



In [ ]:

In [ ]: